

08 — Schema Design: Social Media Platform

Table of Contents

- 1. [Learning Objectives](#)
- 2. [Domain Overview](#)
- 3. [ASCII ER Diagram](#)
- 4. [Complete DDL Script](#)
- 5. [The Follow Graph Problem](#)
- 6. [Feed Generation Strategies](#)
- 7. [Common Queries](#)
- 8. [Scaling Challenges](#)
- 9. [Common Mistakes](#)
- 10. [Best Practices](#)
- 11. [Performance Considerations](#)
- 12. [Interview Questions & Answers](#)
- 13. [Exercises with Solutions](#)
- 14. [Cross-References](#)

Learning Objectives

After completing this section you will be able to:

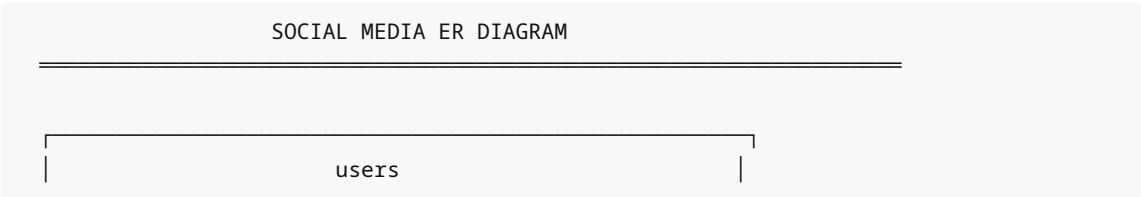
- Design a social graph (follows, friends, blocks)
- Model posts, comments, likes, and media
- Implement feed generation strategies (pull vs. push vs. hybrid)
- Handle the "celebrity problem" in social graphs
- Design notification and messaging systems

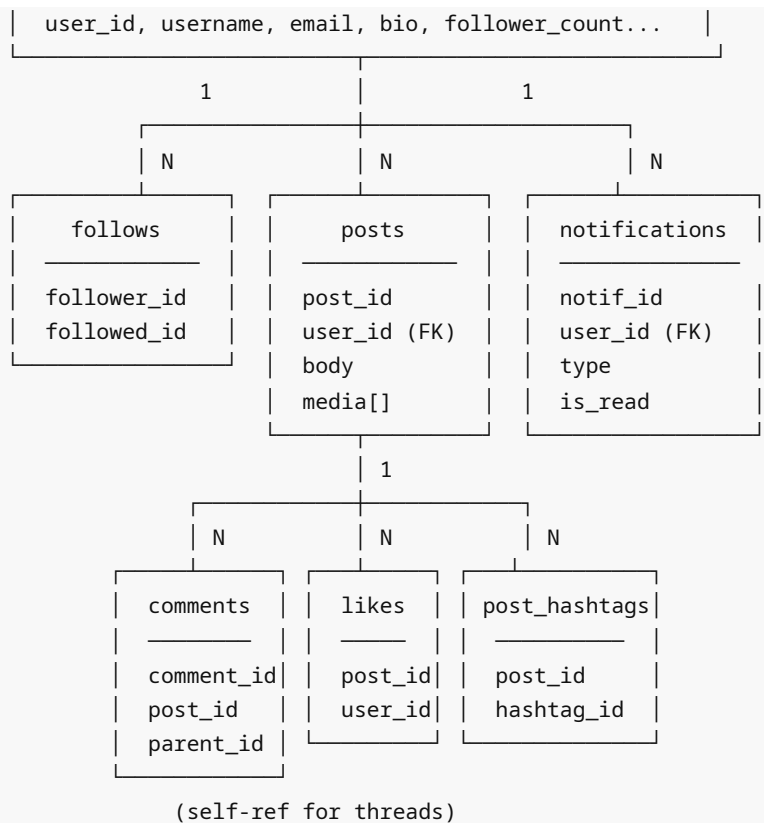
Domain Overview

A social media platform manages:

- **Users & Profiles:** accounts, bios, avatars, verification
- **Social Graph:** follow/friend relationships, blocks
- **Content:** posts, comments, media attachments
- **Reactions:** likes, reposts, bookmarks
- **Feed:** personalized timeline per user
- **Notifications:** system-generated alerts
- **Direct Messages:** private conversations
- **Hashtags & Discovery:** content organization

ASCII ER Diagram





Complete DDL Script

```

-- =====
-- SOCIAL MEDIA PLATFORM SCHEMA
-- =====

-- =====
-- USERS & PROFILES
-- =====

CREATE TABLE users (
    user_id          BIGINT          GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    username         VARCHAR(30) NOT NULL UNIQUE
                        CHECK (username ~ '^[a-zA-Z0-9_]{3,30}$'),
    email            TEXT            NOT NULL UNIQUE
                        CHECK (email ~ '^[^@\\s]+@[^@\\s]+\\.([^@\\s]+)$'),
    password_hash    TEXT            NOT NULL,
    display_name     TEXT            NOT NULL CHECK (length(trim(display_name)) >= 1),
    bio              TEXT            CHECK (length(bio) <= 500),
    avatar_url       TEXT,
    header_url       TEXT,
    website          TEXT,
    location         TEXT,
    date_of_birth    DATE,

```

```

-- Verification
is_verified      BOOLEAN      NOT NULL DEFAULT FALSE, -- blue checkmark
is_active        BOOLEAN      NOT NULL DEFAULT TRUE,
is_private       BOOLEAN      NOT NULL DEFAULT FALSE, -- private account
-- Counter caches (denormalized for performance)
follower_count   INTEGER      NOT NULL DEFAULT 0 CHECK (follower_count >= 0),
following_count  INTEGER      NOT NULL DEFAULT 0 CHECK (following_count >= 0),
post_count       INTEGER      NOT NULL DEFAULT 0 CHECK (post_count >= 0),
-- Timestamps
created_at       TIMESTAMPTZ  NOT NULL DEFAULT now(),
updated_at       TIMESTAMPTZ  NOT NULL DEFAULT now()
);

CREATE INDEX idx_users_username ON users (username);
CREATE INDEX idx_users_email   ON users (email);

-- =====
-- SOCIAL GRAPH
-- =====

-- Directed follow graph (user A follows user B)
CREATE TABLE follows (
    follower_id    BIGINT      NOT NULL REFERENCES users(user_id) ON DELETE
CASCADE,
    followed_id    BIGINT      NOT NULL REFERENCES users(user_id) ON DELETE
CASCADE,
    created_at     TIMESTAMPTZ  NOT NULL DEFAULT now(),
    PRIMARY KEY (follower_id, followed_id),
    CONSTRAINT no_self_follow CHECK (follower_id != followed_id)
);

CREATE INDEX idx_follows_followed ON follows (followed_id, created_at DESC); --
"who follows me?"
CREATE INDEX idx_follows_follower ON follows (follower_id, created_at DESC); --
"who do I follow?"

-- Blocks: users can block others
CREATE TABLE blocks (
    blocker_id     BIGINT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    blocked_id     BIGINT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    created_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (blocker_id, blocked_id),
    CONSTRAINT no_self_block CHECK (blocker_id != blocked_id)
);

CREATE INDEX idx_blocks_blocked ON blocks (blocked_id);

-- Pending follow requests (for private accounts)
CREATE TABLE follow_requests (
    requester_id   BIGINT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    requested_id   BIGINT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    requested_at   TIMESTAMPTZ NOT NULL DEFAULT now(),

```

```

        status          TEXT NOT NULL DEFAULT 'pending'
                        CHECK (status IN ('pending', 'approved', 'rejected')),
        PRIMARY KEY (requester_id, requested_id)
);

-- =====
-- CONTENT: POSTS
-- =====

CREATE TABLE posts (
    post_id            BIGINT      GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    user_id            BIGINT      NOT NULL REFERENCES users(user_id) ON DELETE
CASCADE,
    body               TEXT        CHECK (length(body) <= 5000),
    post_type          TEXT        NOT NULL DEFAULT 'post'
                        CHECK (post_type IN ('post', 'repost', 'reply', 'quote')),
    -- For reposts and replies
    parent_post_id     BIGINT      REFERENCES posts(post_id) ON DELETE SET NULL,
    quoted_post_id     BIGINT      REFERENCES posts(post_id) ON DELETE SET NULL,
    -- Visibility
    visibility          TEXT        NOT NULL DEFAULT 'public'
                        CHECK (visibility IN
('public', 'followers', 'mentioned', 'private')),
    is_sensitive        BOOLEAN     NOT NULL DEFAULT FALSE,
    -- Counter caches
    like_count          INTEGER     NOT NULL DEFAULT 0 CHECK (like_count >= 0),
    repost_count        INTEGER     NOT NULL DEFAULT 0 CHECK (repost_count >= 0),
    reply_count         INTEGER     NOT NULL DEFAULT 0 CHECK (reply_count >= 0),
    bookmark_count      INTEGER     NOT NULL DEFAULT 0 CHECK (bookmark_count >= 0),
    -- Search
    search_vec          TSVECTOR    GENERATED ALWAYS AS (
                                to_tsvector('english', coalesce(body, ''))
                                ) STORED,
    -- Timestamps
    created_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
    deleted_at          TIMESTAMPTZ, -- soft delete
    CONSTRAINT body_or_media CHECK (body IS NOT NULL OR post_type = 'repost')
);

CREATE INDEX idx_posts_user          ON posts (user_id, created_at DESC);
CREATE INDEX idx_posts_created      ON posts (created_at DESC) WHERE deleted_at IS
NULL;
CREATE INDEX idx_posts_parent        ON posts (parent_post_id) WHERE parent_post_id IS
NOT NULL;
CREATE INDEX idx_posts_search        ON posts USING GIN (search_vec);
CREATE INDEX idx_posts_public        ON posts (created_at DESC)
    WHERE visibility = 'public' AND deleted_at IS NULL;

-- Media attachments
CREATE TABLE post_media (
    media_id           BIGINT      GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

```

```

        post_id    BIGINT NOT NULL REFERENCES posts(post_id) ON DELETE CASCADE,
        url        TEXT    NOT NULL,
        media_type TEXT    NOT NULL CHECK (media_type IN
('image','video','gif','audio')),
        width      INTEGER,
        height     INTEGER,
        duration_ms INTEGER, -- for video/audio
        alt_text   TEXT,
        display_order SMALLINT NOT NULL DEFAULT 0,
        created_at TIMESTAMPTZ NOT NULL DEFAULT now()
    );

CREATE INDEX idx_media_post ON post_media (post_id);

-- =====
-- REACTIONS
-- =====

CREATE TABLE likes (
    user_id    BIGINT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    post_id    BIGINT NOT NULL REFERENCES posts(post_id) ON DELETE CASCADE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (user_id, post_id)
);

CREATE INDEX idx_likes_post ON likes (post_id);

CREATE TABLE bookmarks (
    user_id    BIGINT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    post_id    BIGINT NOT NULL REFERENCES posts(post_id) ON DELETE CASCADE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (user_id, post_id)
);

CREATE INDEX idx_bookmarks_user ON bookmarks (user_id, created_at DESC);

-- =====
-- COMMENTS (threaded)
-- =====

CREATE TABLE comments (
    comment_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    post_id    BIGINT NOT NULL REFERENCES posts(post_id) ON DELETE CASCADE,
    user_id    BIGINT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    parent_id  BIGINT REFERENCES comments(comment_id) ON DELETE CASCADE, --
threading
    body       TEXT    NOT NULL CHECK (length(trim(body)) >= 1 AND length(body) <=
2000),
    like_count INTEGER NOT NULL DEFAULT 0 CHECK (like_count >= 0),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    deleted_at TIMESTAMPTZ
);

```

```

CREATE INDEX idx_comments_post    ON comments (post_id, created_at);
CREATE INDEX idx_comments_parent ON comments (parent_id) WHERE parent_id IS NOT
NULL;

-- =====
-- HASHTAGS
-- =====

CREATE TABLE hashtags (
  hashtag_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  tag        TEXT    NOT NULL UNIQUE CHECK (tag ~ '^[a-zA-Z0-9_]{1,100}$'),
  post_count INTEGER NOT NULL DEFAULT 0,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE post_hashtags (
  post_id    BIGINT NOT NULL REFERENCES posts(post_id) ON DELETE CASCADE,
  hashtag_id INTEGER NOT NULL REFERENCES hashtags(hashtag_id) ON DELETE CASCADE,
  PRIMARY KEY (post_id, hashtag_id)
);

CREATE INDEX idx_post_hashtags_tag ON post_hashtags (hashtag_id);

-- =====
-- MENTIONS
-- =====

CREATE TABLE mentions (
  mention_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  post_id    BIGINT NOT NULL REFERENCES posts(post_id) ON DELETE CASCADE,
  mentioned_user_id BIGINT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  UNIQUE (post_id, mentioned_user_id)
);

CREATE INDEX idx_mentions_user ON mentions (mentioned_user_id);

-- =====
-- NOTIFICATIONS
-- =====

CREATE TABLE notifications (
  notif_id    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  user_id     BIGINT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  actor_id    BIGINT REFERENCES users(user_id) ON DELETE SET NULL,
  notif_type  TEXT    NOT NULL
                        CHECK (notif_type IN ('like', 'repost', 'follow', 'mention',
                                              'reply', 'dm', 'system')),
  post_id     BIGINT REFERENCES posts(post_id) ON DELETE CASCADE,
  -- Aggregated: "Alice and 5 others liked your post"
  aggregate_count INTEGER NOT NULL DEFAULT 1,
  is_read     BOOLEAN NOT NULL DEFAULT FALSE,

```

```

        created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
        read_at         TIMESTAMPTZ
    );

CREATE INDEX idx_notifs_user_unread ON notifications (user_id, created_at DESC)
    WHERE is_read = FALSE;

-- =====
-- DIRECT MESSAGES
-- =====

CREATE TABLE conversations (
    conversation_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    last_message_at TIMESTAMPTZ
);

CREATE TABLE conversation_participants (
    conversation_id BIGINT NOT NULL REFERENCES conversations(conversation_id) ON
DELETE CASCADE,
    user_id         BIGINT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    last_read_at    TIMESTAMPTZ,
    PRIMARY KEY (conversation_id, user_id)
);

CREATE INDEX idx_conv_participants_user ON conversation_participants (user_id,
conversation_id);

CREATE TABLE messages (
    message_id      BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    conversation_id BIGINT NOT NULL REFERENCES conversations(conversation_id),
    sender_id       BIGINT NOT NULL REFERENCES users(user_id),
    body            TEXT NOT NULL CHECK (length(body) <= 10000),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    deleted_at      TIMESTAMPTZ
);

CREATE INDEX idx_messages_conversation ON messages (conversation_id, created_at
DESC);

-- =====
-- FEED TABLE (push fan-out model)
-- =====

CREATE TABLE feeds (
    user_id         BIGINT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    post_id         BIGINT NOT NULL REFERENCES posts(post_id) ON DELETE CASCADE,
    post_score      REAL NOT NULL DEFAULT 0, -- algorithm score
    added_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (user_id, post_id)
);

```

```
CREATE INDEX idx_feeds_user ON feeds (user_id, post_score DESC, added_at DESC);
```

The Follow Graph Problem

Fan-out (push model)

When user A posts, their post is pushed to all followers' feeds.

User A (10M followers) posts → push to 10M feeds
Problem: 10M writes per post for "celebrities"
Solution: Hybrid model – push for normal users, pull for celebrities

Pull model

```
-- Pull model: compute feed at read time
SELECT p.*
FROM posts p
JOIN follows f ON f.followed_id = p.user_id
WHERE f.follower_id = :current_user
      AND p.created_at > now() - INTERVAL '7 days'
      AND p.deleted_at IS NULL
ORDER BY p.created_at DESC
LIMIT 20;
-- Problem: slow for users following thousands of people
```

Hybrid model

```
-- Push feed entries for normal users (< 1M followers)
-- Pull for celebrity posts (> 1M followers) and merge

CREATE OR REPLACE FUNCTION get_feed(p_user_id BIGINT, p_limit INT DEFAULT 20)
RETURNS TABLE (post_id BIGINT, score REAL)
LANGUAGE SQL STABLE AS $$
    -- Pre-computed fan-out feed entries
    SELECT post_id, post_score
    FROM feeds
    WHERE user_id = p_user_id

    UNION ALL

    -- Real-time pull for followed celebrities
    SELECT p.post_id, 1.0 AS score
    FROM posts p
    JOIN follows f ON f.followed_id = p.user_id AND f.follower_id = p_user_id
    JOIN users u ON u.user_id = p.user_id
    WHERE u.follower_count > 1000000 -- celebrity threshold
          AND p.created_at > now() - INTERVAL '7 days'
          AND p.deleted_at IS NULL
```



```
ORDER BY score DESC, post_id DESC
LIMIT p_limit;
$;
```

Common Queries

```
-- User's timeline (posts they created)
SELECT p.*, u.username, u.avatar_url
FROM posts p
JOIN users u ON u.user_id = p.user_id
WHERE p.user_id = :user_id AND p.deleted_at IS NULL
ORDER BY p.created_at DESC LIMIT 20;

-- Mutual follows (people we both follow)
SELECT u.*
FROM follows f1
JOIN follows f2 ON f2.follower_id = :other_user AND f2.followed_id = f1.followed_id
JOIN users u ON u.user_id = f1.followed_id
WHERE f1.follower_id = :me AND f1.followed_id != :other_user;

-- Trending hashtags (last 24h)
SELECT h.tag, count(*) AS recent_posts
FROM post_hashtags ph
JOIN hashtags h ON h.hashtag_id = ph.hashtag_id
JOIN posts p ON p.post_id = ph.post_id
WHERE p.created_at >= now() - INTERVAL '24 hours'
AND p.deleted_at IS NULL
GROUP BY h.hashtag_id, h.tag
ORDER BY recent_posts DESC LIMIT 10;

-- Thread of replies
WITH RECURSIVE reply_thread AS (
    SELECT comment_id, parent_id, user_id, body, created_at, 0 AS depth
    FROM comments
    WHERE post_id = :post_id AND parent_id IS NULL

    UNION ALL

    SELECT c.comment_id, c.parent_id, c.user_id, c.body, c.created_at, rt.depth + 1
    FROM comments c
    JOIN reply_thread rt ON rt.comment_id = c.parent_id
    WHERE c.deleted_at IS NULL AND rt.depth < 5 -- max depth limit
)
SELECT rt.*, u.username FROM reply_thread rt
JOIN users u ON u.user_id = rt.user_id
ORDER BY depth, created_at;
```

Common Mistakes

1. **Not caching follower/following counts** — `COUNT(*)` on the follows table is expensive at scale. Use counter caches.
 2. **Naive feed query with large follow lists** — Joining posts with a user's 1000 follows without index and limit is a disaster.
 3. **Storing full post body in feeds table** — Store only `post_id` in feeds; join to posts at read time.
 4. **No soft delete** — Hard-deleting posts breaks foreign keys and removes audit trail. Use `deleted_at`.
 5. **No block filtering in feed** — Feed queries must exclude blocked users' content.
-

Best Practices

1. **Use counter caches** for `follower_count`, `like_count`, etc.
 2. **Soft delete posts** with `deleted_at`.
 3. **Add block filtering** to all feed/content queries.
 4. **Use UUIDs or BIGINT** for public-facing IDs (BIGINT is sequential — exposure risk).
 5. **Paginate with cursor** (`last_seen_id`), not OFFSET — OFFSET is $O(n)$.
 6. **Implement rate limiting** at the application layer for write operations.
-

Performance Considerations

```
-- Cursor-based pagination (fast, consistent)
SELECT * FROM posts
WHERE user_id = :user_id
      AND post_id < :last_seen_id -- cursor
      AND deleted_at IS NULL
ORDER BY post_id DESC
LIMIT 20;

-- vs OFFSET pagination (slow - must scan and skip)
SELECT * FROM posts
WHERE user_id = :user_id
ORDER BY post_id DESC
LIMIT 20 OFFSET 10000; -- scans 10020 rows to return 20!
```

Interview Questions & Answers

Q1: How would you design a feed system for a social media platform?

A: For small-scale: pull model (query followed users' posts at read time). For medium scale: fan-out push model (write to each follower's feed table on post creation). For large scale: hybrid (push for regular users, pull for celebrities). The choice depends on read/write ratio, average follower count, and latency requirements.

Q2: What is the "celebrity problem" in social media?

A: When a celebrity with millions of followers posts, a naive fan-out would require millions of writes. The hybrid approach handles celebrities differently: their posts are pulled at read time from the timeline rather than pre-written to each follower's feed, while normal users get their posts pushed.

Q3: How do you implement cursor-based pagination?

A: Use a `WHERE id < :last_seen_id ORDER BY id DESC LIMIT N` pattern instead of `OFFSET N`. The `last_seen_id` is the smallest ID from the previous page. This is $O(\log n)$ via B-tree index, while `OFFSET` is $O(n)$ (must skip N rows).

Q4: How would you model threaded comments?

A: Self-referencing `parent_id` column (adjacency list). Use a recursive CTE to fetch threads up to a depth limit. For read-heavy applications, store the full path or use the `ltree` extension for efficient subtree queries.

Q5: Why use soft delete for posts?

A: Hard-deleting posts breaks foreign key references from likes, comments, bookmarks, and notifications. Soft delete (`deleted_at` timestamp) preserves data integrity, allows undo/recovery, maintains analytics accuracy, and satisfies legal hold requirements.

Exercises with Solutions

Exercise 1

Write a query to get a user's feed (posts from people they follow), excluding blocked users, limited to last 7 days, paginated.

Solution:

```
SELECT p.post_id, p.body, p.created_at, p.like_count, u.username, u.avatar_url
FROM posts p
JOIN follows f ON f.followed_id = p.user_id AND f.follower_id = :current_user
JOIN users u ON u.user_id = p.user_id
WHERE p.deleted_at IS NULL
      AND p.created_at >= now() - INTERVAL '7 days'
      AND p.visibility IN ('public', 'followers')
      AND p.user_id NOT IN (
        SELECT blocked_id FROM blocks WHERE blocker_id = :current_user
        UNION
        SELECT blocker_id FROM blocks WHERE blocked_id = :current_user
      )
      AND p.post_id < :cursor_post_id -- cursor pagination
ORDER BY p.post_id DESC
LIMIT 20;
```

Cross-References

- `04_entity_relationships.md` — self-referencing FK (follows, comments)
- `03_denormalization.md` — counter caches for likes, follows
- `10_design_patterns.md` — soft deletes, audit trails

- `../05_PostgreSQL_Core/06_array_types.md` — hashtag arrays alternative